Università di Padova

Dipartimento di Matematica "Tullio Levi-Civita"

Corso di Laurea in Informatica



ChatCart: agente conversazionale con interfaccia generativa

Tesi di Laurea

Laureando

Edoardo De Piccoli

Matricola 2101055

Relatore

Prof. Tullio Vardanega

Anno Accademico 2025-2026

© Edoardo De Piccoli. Tutti i diritti riservati. Tesi di Laurea: “ChatCart: agente conversazionale con interfaccia generativa”, Università di Padova, Dipartimento di Matematica “Tullio Levi-Civita”, Corso di Laurea in Informatica.

Sommario

In questa relazione descrivo lo *stage* che ho svolto presso Halue S.r.l., società di consulenza tecnologica con sede operativa ad Abano Terme, dal 14 settembre al 9 novembre 2026.

Con la supervisione del Prof. Tullio Vardanega, relatore, e di Antonio Cavestro, *tutor* aziendale, ho progettato e realizzato ChatCart, il prototipo di un assistente conversazionale per i negozi Shopify che risponde al cliente non soltanto con testo, ma anche con componenti dell'interfaccia che sceglie e popola in tempo reale con i dati del negozio.

Struttura del documento. La relazione si articola in quattro capitoli:

- **Capitolo 1: «Halue e il commercio digitale».** Presento l'azienda, la sua clientela, le tecnologie e i processi che adotta e il suo rapporto con l'innovazione, in particolare con l'intelligenza artificiale.
- **Capitolo 2: «Oltre la conversazione testuale».** Discuto il problema che lo *stage* affronta, il suo posto nella strategia di Halue, gli obiettivi e i vincoli del progetto e le ragioni per cui l'ho scelto.
- **Capitolo 3: «Progetto e sviluppo di ChatCart».** Illustro il metodo di lavoro, l'analisi, l'architettura, la realizzazione dell'agente e del *widget*, le funzionalità sperimentali e le possibili evoluzioni del progetto, la verifica e i risultati ottenuti.
- **Capitolo 4: «Valutazione retrospettiva».** Valuto il grado di soddisfacimento degli obiettivi, la mia maturazione professionale e la distanza tra le competenze che lo *stage* mi ha richiesto e quelle che ho acquisito nel corso di studi.

In appendice raccolgo gli acronimi e le abbreviazioni con la loro forma estesa, il glossario e i riferimenti.

Convenzioni tipografiche. Nel documento adotto le seguenti convenzioni:

- il corsivo indica i termini in lingua straniera, e soltanto questi; i nomi propri di aziende, prodotti e metodi (come Shopify, Slack o Scrum) restano in tondo;
- alla prima occorrenza di un termine presente nel glossario ne do nel testo una definizione essenziale e lo contrassegno con il pedice G; il termine e il pedice sono cliccabili e rimandano all'approfondimento nel glossario;
- figure, tabelle e codici sorgente hanno una numerazione progressiva e una didascalia posta sotto l'elemento, che ne indica la fonte quando non sono di mia realizzazione.

Indice

1 Halue e il commercio digitale **1**

1.1 Profilo aziendale e linee di attività 1

1.2 Clientela e modalità di ingaggio 4

1.3 Ecosistema tecnologico 6

1.4 Processi di sviluppo e gestione dei progetti 7

1.4.1 Organizzazione del lavoro e gestione dei progetti 7

1.4.2 Dal *ticket* al rilascio 8

1.5 Innovazione e intelligenza artificiale 10

2 Oltre la conversazione testuale **12**

2.1 Il ruolo degli *stage* nella strategia aziendale 12

2.2 I limiti degli assistenti conversazionali testuali 15

2.3 Il commercio agentico nella visione di Halue 17

2.4 Obiettivi e vincoli del progetto 20

2.5 Motivazioni e obiettivi personali 23

3 Progetto e sviluppo di ChatCart **26**

3.1 Metodo di lavoro 26

3.1.1 Pianificazione e interazione con il *tutor* aziendale 26

3.1.2 Organizzazione del lavoro e tracciamento delle attività 26

3.1.3 Strumenti e assistenti di programmazione 26

3.2 Studio preliminare e analisi 27

3.2.1 L'ecosistema delle applicazioni Shopify 27

3.2.2 Un'analisi dei requisiti iterativa 27

3.3 Architettura del sistema 27

3.3.1 Visione d'insieme 27

3.3.2 Il contratto tra agente e *widget* 28

3.4 Realizzazione 28
3.4.1 L'agente conversazionale 28
3.4.2 Il *widget* e l'integrazione con il tema del negozio 28
3.4.3 Metriche di utilizzo e analisi delle conversazioni 28
3.5 Sperimentazioni e sviluppi futuri 29
3.6 Verifica e validazione 29
3.7 Risultati 29
3.7.1 Il prodotto dal punto di vista dell'utente 29
3.7.2 Risultati quantitativi 29

4 Valutazione retrospettiva **30**
4.1 Soddisfacimento degli obiettivi 30
4.2 Maturazione professionale 30
4.3 Formazione universitaria e lavoro 31

Acronimi e abbreviazioni **32**

Glossario **34**

Riferimenti **40**

Elenco delle figure

1.1 Le linee di attività di Halue 2
1.2 Le integrazioni di un negozio Shopify 4
1.3 Esempio di architettura di un progetto di intelligenza artificiale. . . . 7
1.4 Il ciclo di vita di un *ticket* in Halue, come diagramma di attività UML. 9
1.5 La quota del tempo che Halue dedica alla ricerca e sviluppo. 10
1.6 Il ciclo di lavoro con gli strumenti di intelligenza artificiale in Halue. . 10

2.1 Un flusso di conversazione predefinito e il ciclo di un agente 13
2.2 Confronto tra un assistente testuale e ChatCart 15
2.3 I tre approcci all'interfaccia generativa 18
2.4 Le due direzioni del commercio agentico 19
2.5 L'architettura su due livelli prevista dal piano di lavoro 20
2.6 La ripartizione delle ore prevista dal piano di lavoro 23

Elenco delle tabelle

1.1 Gli strumenti di coordinamento del *team* di Halue. 8
1.2 Le pratiche di Halue lungo il ciclo di un *ticket*. 9

2.1 Gli obiettivi obbligatori del piano di lavoro. 21
2.2 Gli obiettivi desiderabili e facoltativi del piano di lavoro. 21
2.3 I miei obiettivi personali per lo *stage*. 25

Elenco dei codici sorgente

1 Halue e il commercio digitale

Presento Halue S.r.l., l'azienda presso cui ho svolto lo *stage*, descrivendone le attività, i clienti, le tecnologie, i processi e il rapporto con l'innovazione. Il quadro nasce dall'esperienza diretta delle otto settimane di lavoro, dai colloqui con i colleghi, da un'intervista dedicata al *tutor* aziendale, Antonio Cavestro, e dalle informazioni pubbliche del sito dell'azienda[1].

1.1 Profilo aziendale e linee di attività

Halue è una società di consulenza tecnologica nata nel 2022, con sede legale a Parma e sede operativa ad Abano Terme, in provincia di Padova. È nata come *boutique* di consulenza, cioè una società piccola e specializzata in pochi ambiti, sull'*e-commerce*$_{\text{G}}$ e sui sistemi di *Customer Relationship Management* (CRM)$_{\text{G}}$, con Shopify e Salesforce come piattaforme di riferimento. Il primo ambito riguarda la vendita di prodotti e servizi tramite canali digitali. Il secondo riguarda la gestione delle relazioni delle aziende con i propri clienti, dalle vendite all'assistenza al *marketing*.

Negli anni successivi Halue ha esteso la propria attività all'intelligenza artificiale, a partire dai progetti stessi. I negozi dei suoi clienti hanno cataloghi ampi, contenuti in più lingue e percorsi d'acquisto complessi. In questi punti un modello linguistico collegato ai dati del negozio aiuta sia chi acquista sia chi gestisce il negozio. Nello stesso periodo le piattaforme di commercio elettronico hanno iniziato a offrire strumenti e protocolli pensati per gli agenti di intelligenza artificiale, che presento nella sezione 1.3. La figura 1.1 riassume le linee di attività di oggi, che descrivo più avanti in questa sezione.

Dal 2024 Halue è una Società Benefit$_{\text{G}}$, una forma giuridica introdotta in Italia nel 2015 per le società che oltre al profitto perseguono obiettivi di beneficio comune dichiarati nello statuto e ne rendono conto ogni anno in una relazione d'impatto. Gli



Figura 1.1: Le linee di attività di Halue. La consulenza tecnologica fa da cornice a tutte; l'area di ciascun blocco segue l'ordine di peso delle linee, senza riprodurne una misura.

obiettivi di Halue riguardano l'ambiente, le persone e le comunità. L'azienda misura le proprie pratiche con il *B Impact Assessment*$_{G}$, un questionario di valutazione che le esamina in cinque aree: *governance*, lavoratori, clienti, comunità e ambiente. Tra i risultati del 2025, Halue riporta[2]:

- un codice etico e politiche interne sulla segnalazione di illeciti, sugli abusi e sulle discriminazioni;
- linee guida per rendere sostenibile il lavoro da remoto;
- una collaborazione per la tutela della fauna marina nel Delta del Po.

Oggi l'azienda conta dieci persone, con i seguenti ruoli:

- *Founder & Senior Consultant*$_{G}$: il fondatore Valentino Baraldo, che svolge anche attività di consulenza;
- *Solution & Technical Architect*$_{G}$: definisce le soluzioni e l'architettura tecnica;
- *Senior Project Manager*$_{G}$ e *Project Management Assistant*$_{G}$: due ruoli dedicati alla gestione dei progetti;
- *Senior Software Engineer*$_{G}$, *Software Engineer*$_{G}$ e *Junior Developer*$_{G}$: cinque sviluppatori con diversi livelli di esperienza;
- *UX/UI Digital Consultant*$_{G}$: cura la progettazione dell'esperienza utente e dell'interfaccia.

In un'azienda di queste dimensioni ogni ruolo copre un'intera fase del progetto: la consulenza iniziale, l'architettura, la gestione, lo sviluppo e la progettazione dell'interfaccia. Ogni persona lavora per più clienti nello stesso periodo.

Halue lavora su commessa: costruisce soluzioni su misura per i singoli clienti e non vende prodotti propri, come applicazioni pubblicate sullo Shopify App Store. Le sue competenze crescono quindi progetto dopo progetto, guidate dalle richieste dei clienti, che l'azienda segue da vicino in un rapporto di lavoro continuo, possibile per un'azienda delle sue dimensioni.

La consulenza tecnologica accompagna tutte le linee di attività dell'azienda, dalla valutazione iniziale alla scelta dei sistemi. Le linee sono, in ordine di peso:

1. l'*e-commerce* su Shopify e su Salesforce Commerce Cloud, sia verso il consumatore finale (B2C$_G$) sia verso altre aziende (B2B$_G$). Nel B2B il negozio gestisce anche listini riservati, ordini minimi e condizioni di pagamento per ciascuna azienda cliente;
2. l'integrazione di sistemi, cioè il collegamento del negozio con la logistica, con i servizi di terze parti e con i sistemi gestionali (ERP$_G$), che riuniscono in un'unica base di dati amministrazione, acquisti, magazzino e vendite e con cui il negozio sincronizza ordini e disponibilità dei prodotti. Il collegamento può essere punto a punto, costruito per ogni coppia di sistemi, oppure passare per una piattaforma di integrazione erogata come servizio (iPaaS$_G$), che offre connettori già pronti e strumenti per trasformare i dati;
3. i servizi di intelligenza artificiale, dalla valutazione di quanto un'azienda sia pronta ad adottarla, anche per la qualità e l'organizzazione dei suoi dati, fino allo sviluppo di agenti su misura;
4. i sistemi CRM su Salesforce, compresi i prodotti per l'assistenza ai clienti e per il *marketing*;
5. la formazione, rivolta sia ai clienti sia ai dipendenti. Per i clienti, Halue prepara incontri, come *workshop* formativi sull'intelligenza artificiale, tenuti direttamente dal fondatore, che spiegano il funzionamento dei modelli linguistici e rendono il cliente consapevole delle loro possibilità e dei loro limiti. I dipendenti seguono una formazione interna sui metodi agili, sulle piattaforme e sulla tecnologia, e percorsi personali per crescere come professionisti, anche oltre il loro ruolo in azienda;
6. la progettazione dell'interfaccia e dell'esperienza utente.

Le prime due linee si incontrano in quasi ogni progetto: il negozio raramente lavora da solo, e scambia di continuo dati con i sistemi che l'azienda cliente usa già.

Le due piattaforme di *e-commerce* hanno pesato in modo diverso nella storia di Halue. Nei primi anni l'azienda si è specializzata anche su Salesforce Commerce



Figura 1.2: Un esempio di integrazioni punto a punto di un negozio Shopify con CRM, gestionale ERP, logistica e servizi di terze parti. Le finestre sono illustrative; le frecce verso l'esterno indicano i dati che il negozio invia, quelle verso il centro i dati che riceve. Con una piattaforma iPaaS gli stessi flussi passano per un unico punto di raccordo invece che per un collegamento dedicato a ogni sistema.

Cloud, la piattaforma di commercio elettronico della *suite* di Salesforce, e ne segue ancora i clienti. Oggi la specializzazione principale è Shopify Plus, che presento nella sezione 1.2.

L'intelligenza artificiale è insieme una linea a sé e, sempre di più, una competenza che entra nei progetti di *e-commerce* e di integrazione.

1.2 Clientela e modalità di ingaggio

Halue lavora soltanto con aziende private, per lo più aziende italiane di medie e grandi dimensioni, e non con la pubblica amministrazione. Sono marchi strutturati della moda, dello sport, del commercio al dettaglio e della cosmetica professionale, con fatturati che vanno da decine a diverse centinaia di milioni di euro all'anno, e vendono *online* su più mercati, con lingue e valute diverse.

Vendere su più mercati moltiplica il lavoro tecnico. Ogni mercato ha la sua lingua, la sua valuta, i suoi prezzi e le sue tasse, spesso anche un magazzino proprio, e ogni negozio deve restare allineato con il sistema gestionale dell'azienda. Per questo molti clienti di Halue usano Shopify Plus$_{G}$, la proposta di Shopify per le aziende di grandi dimensioni, che aggiunge al negozio, tra l'altro, funzioni dedicate alla vendita B2B[3].

Alcuni clienti hanno decine di negozi attivi, uno per mercato o per canale, ciascuno con il proprio catalogo e il proprio magazzino.

Halue è *Shopify Plus Partner* dal 2023. Il programma partner di Shopify assegna alle agenzie un livello in base ai negozi che hanno avviato o trasferito sulla piattaforma e al volume di vendite di quei negozi. Il livello dà all'agenzia visibilità nell'elenco ufficiale dei partner, che i commercianti consultano per scegliere a chi affidare il proprio negozio. Nel suo profilo Halue indica come specializzazioni, tra le altre, lo sviluppo di applicazioni e API su misura, la personalizzazione della pagina di pagamento e dell'area personale dei clienti e l'ottimizzazione per i motori di ricerca[4].

Alcune delle richieste dei clienti che ho potuto notare in prima persona durante le riunioni quotidiane con il *team* sono:

- la migrazione di un negozio verso Shopify da altre piattaforme, come Salesforce Commerce Cloud, WooCommerce e Magento. Oltre a catalogo, clienti e ordini, la migrazione trasferisce le integrazioni con i sistemi esterni e conserva gli indirizzi delle pagine, perché i collegamenti esistenti e il posizionamento nei motori di ricerca non vadano persi;
- il *redesign*$_{\text{G}}$ di un negozio esistente;
- lo sviluppo di interfacce su misura, con *landing page*$_{\text{G}}$ dinamiche (pagine che accolgono chi arriva da una campagna o da un annuncio e lo guidano verso un acquisto, con contenuti che cambiano in base al mercato o al visitatore) e configuratori 3D;
- l'espansione su nuovi mercati e la manutenzione periodica dei negozi in produzione, con aggiornamenti di configurazione, correzioni ed evoluzioni.

I progetti arrivano soprattutto per passaparola, da clienti e contatti che segnalano l'azienda, e in parte da gare e bandi. Le formule contrattuali sono tre, ciascuna adatta a un tipo di lavoro:

- a corpo, con un prezzo fissato in anticipo, per i progetti dal perimetro ben definito. Questa formula richiede una stima accurata prima di iniziare, e nella sezione 1.4.2 descrivo il metodo con cui Halue la ottiene;
- a tempo e materiali, per i lavori il cui perimetro si chiarisce strada facendo. Il cliente paga il tempo impiegato, calcolato sul costo giornaliero di ciascun dipendente moltiplicato per il numero dei dipendenti coinvolti e per le giornate di lavoro;
- i contratti di manutenzione e assistenza per i negozi già in produzione, che coprono le segnalazioni e le evoluzioni dopo la consegna.

Le modifiche chieste a progetto avviato diventano *change request* e seguono lo stesso percorso di stima del progetto iniziale. Il rapporto con un cliente prosegue quindi oltre la consegna, e con alcuni clienti è diventato una collaborazione duratura, comprensiva di più progetti successivi alla prima collaborazione con l'azienda.

Il fondatore e il *Senior Project Manager* sono le figure che seguono i clienti con più continuità. Quando un cliente collabora da tempo con uno sviluppatore, per esempio un *Senior Software Engineer*, le richieste di nuove funzionalità e gli aggiornamenti sull'avanzamento passano direttamente da quella persona, nelle *call* e nel canale Slack dedicato al marchio.

1.3 Ecosistema tecnologico

Le interfacce dei negozi Shopify rivolte ai clienti, dette *storefront*$_G$, si possono sviluppare in due modi:

- con un tema della piattaforma: Shopify genera le pagine a partire da modelli scritti in Liquid$_G$, il suo linguaggio di *templating*, e dai dati del negozio;
- con un'architettura *headless*$_G$: l'interfaccia è un'applicazione separata, costruita per esempio con Hydrogen$_G$, il *framework* di Shopify basato su React, e accede al negozio tramite API$_G$, cioè interfacce con cui un programma mette a disposizione di altri programmi dati e operazioni, secondo un contratto definito.

Shopify offre due famiglie di API: le Admin API, per gestire i dati e le operazioni del negozio, e le Storefront API, per catalogo, carrello e acquisto. Entrambe usano GraphQL$_G$, un linguaggio di interrogazione in cui il *client* chiede con una sola richiesta esattamente i campi che gli servono. Un tema Liquid riceve invece i dati del negozio durante la generazione delle pagine. Per aggiungere funzionalità, Halue sviluppa applicazioni personalizzate in Node.js e TypeScript, integrabili con entrambe le modalità.

Nei progetti di intelligenza artificiale cambia soprattutto l'architettura. Al centro c'è un modello linguistico di grandi dimensioni (LLM)$_G$, addestrato a elaborare e generare testo, che chiede all'agente che lo ospita di invocare uno strumento, ne riceve il risultato e ripete il ciclo finché non formula la risposta (figura 1.3). Gli strumenti si possono scrivere su misura per il progetto oppure ottenere tramite MCP$_G$, un protocollo aperto con cui un *server* espone strumenti e risorse, come il catalogo

di un negozio, a qualunque agente lo supporti, senza un'integrazione dedicata per ciascuno.

In ChatCart ho sviluppato una ricerca nel catalogo su misura al posto del *server* MCP di Shopify: da questa scelta nasce il contratto tra agente e *widget* che descrivo nella sezione 3.3.2.

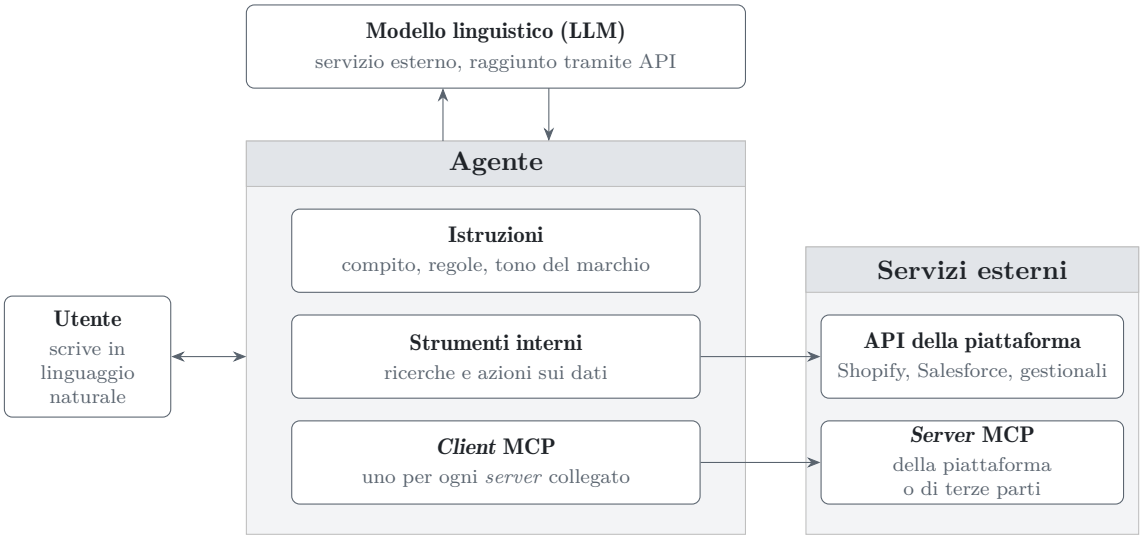


Figura 1.3: Esempio di architettura di un progetto di intelligenza artificiale.

1.4 Processi di sviluppo e gestione dei progetti

1.4.1 Organizzazione del lavoro e gestione dei progetti

Il metodo di riferimento è Scrum$_G$, un *framework* agile per lo sviluppo iterativo, adattato al lavoro di un'agenzia. Il lavoro procede per *sprint*$_G$, cicli brevi al termine dei quali il *team* presenta i risultati al cliente in una *sprint review* e ne discute internamente in una retrospettiva. Mancano le figure canoniche di Scrum, il *Product Owner* e lo *Scrum Master*, e le loro responsabilità passano a due ruoli:

- il *team lead*$_G$, che cura l'analisi funzionale, le stime per il cliente e il coordinamento tecnico;
- il *project/service manager*$_G$, che pianifica le attività e la capacità del *team*.

Accanto a loro lavorano gli sviluppatori e chi progetta l'interfaccia. In un'agenzia lo stesso gruppo di persone lavora per più clienti nello stesso periodo, perciò nessuno è assegnato a un progetto in modo stabile: ogni mese un *demand planning* condiviso ripartisce il *team* sui clienti, e un rapporto sullo stato di avanzamento dei lavori

(SAL)$_G$ riporta con un semaforo lo stato di ciascun progetto, le scadenze, i rischi e i punti aperti.

Il *team* si coordina con gli strumenti della tabella 1.1 e con uno *stand-up*$_G$ quotidiano, una riunione mattutina di quindici-venti minuti in cui ciascuno espone su cosa ha lavorato il giorno prima e su cosa lavorerà quel giorno.

Strumento	**Uso**
Google Meet	riunioni a distanza, tra cui lo *stand-up*
Jira	registro delle richieste e *board* di avanzamento dei progetti
Slack	comunicazione con i clienti, in un canale per ciascun marchio
Email	resoconti delle attività e variazioni contrattuali
Confluence e Google Drive	documentazione e conoscenza interna

Tabella 1.1: Gli strumenti di coordinamento del *team* di Halue.

Gli orari e il luogo di lavoro sono flessibili: i dipendenti lavorano in sede indicativamente tra il 30% e il 40% del tempo e da remoto per il resto, e ognuno gestisce la propria presenza in autonomia. L'inserimento di un nuovo membro del *team* parte dalla documentazione e dalle procedure interne e prosegue con piccoli *ticket*$_G$, le unità di lavoro che Jira registra con un identificativo, una descrizione e uno stato, su un progetto reale e con l'affiancamento di un collega.

1.4.2 Dal *ticket* al rilascio

La figura 1.4 mostra il percorso di una richiesta del cliente, dal *ticket* su Jira alla manutenzione.

Gli *Epic* raccolgono insiemi di funzionalità, le *Story* descrivono singole funzionalità, le *Change Request* le modifiche chieste a progetto avviato e i *Bug* i difetti.

La tabella 1.2 raccoglie, nell'ordine della figura 1.4, le pratiche che accompagnano il ciclo. Il metodo è rigido dove la struttura si ripaga sempre, cioè nella tracciabilità dalla richiesta al codice e nelle stime, e flessibile dove i clienti sono diversi tra loro, come la copertura dei *test* o i livelli di servizio, che si decidono progetto per progetto. La stessa impostazione ha guidato il mio *stage*, per esempio nella scelta di concentrare i *test* automatici solo dove servivano (sezione 3.6).

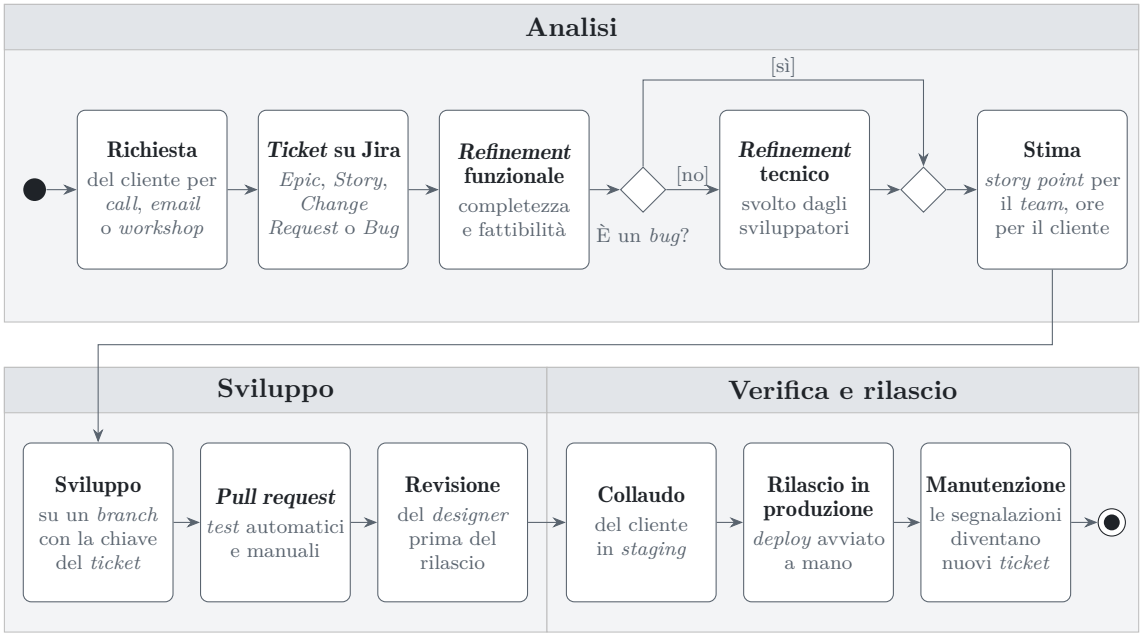


Figura 1.4: Il ciclo di vita di un *ticket* in Halue, come diagramma di attività UML.

Ambito	Pratica
Stima per il *team*	*story point*$_{G}$, una misura della complessità relativa sui numeri di Fibonacci da 1 a 13; oltre 13 il *ticket* si divide
Stima per il cliente	ore, con un metodo interno che pesa complessità, chiarezza della richiesta, *team* coinvolto e confidenza nella stima
Decisioni di architettura	scritte nel *ticket*, oppure in un ADR$_{G}$, un documento breve con contesto, scelta e motivazione, se valgono per tutto il progetto
Progetto grafico	da Figma, collegato al *ticket*, in versione per dispositivi mobili e per *desktop*
Codice	una *pull request*$_{G}$, cioè una proposta di integrazione delle modifiche, per ogni *ticket*, con la sua chiave nei messaggi di *commit*; stile controllato da ESLint e Prettier
Versioni	versionamento semantico (SemVer)$_{G}$, che distingue modifiche incompatibili, nuove funzionalità e correzioni, con *tag* e *release* su GitHub
Test	nessuna copertura minima prefissata
Collaudo	in *staging*$_{G}$ o *preview*, un ambiente separato che riproduce le condizioni di produzione
Deploy	automatizzato, secondo una procedura scritta
Manutenzione	segnalazioni da Jira, Slack o *email*; priorità concordate caso per caso, senza livelli di servizio formali
Retrospettive	anche alla chiusura di un progetto o di una *release* importante

Tabella 1.2: Le pratiche di Halue lungo il ciclo di un *ticket*.

1.5 Innovazione e intelligenza artificiale

Halue pratica uno *scouting* tecnologico continuo e dedica una quota del tempo a progetti di ricerca e sviluppo, tra cui quelli di intelligenza artificiale (figura 1.5). I dipendenti dedicano allo studio una parte rilevante del proprio tempo.

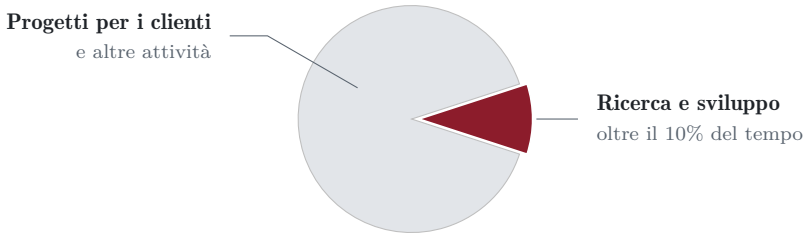


Figura 1.5: La quota del tempo che Halue dedica alla ricerca e sviluppo.

L'intelligenza artificiale è il campo in cui questa propensione si vede di più. Il *team* segue da vicino le novità del settore, testa regolarmente nuovi strumenti di automazione basati sull'IA e li porta nel lavoro di ogni giorno. Nello sviluppo l'IA esegue una prima revisione automatica del codice e analizza i *changelog* delle tecnologie in uso. Per i processi ricorrenti l'azienda usa agenti di IA guidati da *skill*$_{\text{G}}$ interne, cioè documenti di istruzioni, a volte accompagnati da *script* e modelli, che descrivono come svolgere un processo e che l'agente consulta quando il compito lo richiede. In tutti questi usi vale la stessa regola: l'IA propone e la persona decide (figura 1.6).

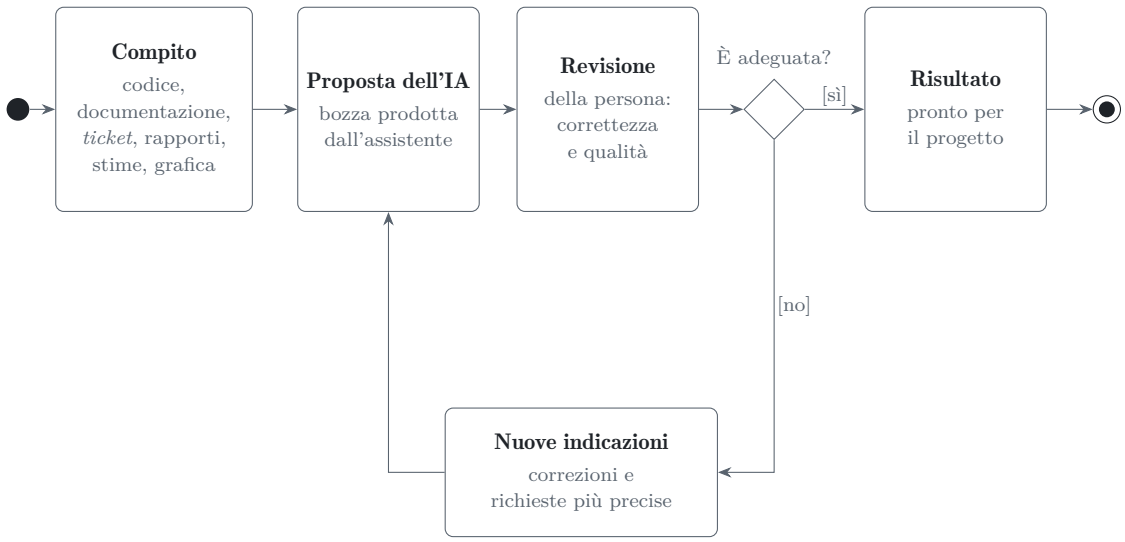


Figura 1.6: Il ciclo di lavoro con gli strumenti di intelligenza artificiale in Halue.

L'azienda riscontra una riduzione dei tempi, ma non dispone ancora di misure precise su questo beneficio e sull'impatto sulla qualità. Nel *team* è condivisa un'opinione su

cui torno nella sezione 4.2: nelle mani di un programmatore esperto l'IA è un buon programmatore *junior*, ma non può ancora sostituire un *senior*.

Nei progetti di IA per i clienti Halue adotta alcune regole che valgono anche per il mio progetto:

- i dati vengono sempre dal catalogo reale, tramite strumenti e ricerca, senza affidarsi alla memoria del modello;
- le azioni con effetti esterni restano sotto controllo umano;
- la qualità delle risposte si valuta con *test* manuali su casi d'uso reali;
- il modello si sceglie in base al compito, in modo da tenere sotto controllo il consumo dei *token*$_{\text{G}}$, i frammenti di testo in cui il modello scompone ciò che legge e scrive e su cui si calcola il costo del suo utilizzo.

Questa propensione all'innovazione passa anche dagli *stage*, che Halue usa per esplorare temi che i progetti per i clienti non lasciano il tempo di approfondire. Approfondisco questo ruolo nella sezione 2.1.

2 Oltre la conversazione testuale

Discuto le ragioni per cui Halue ha proposto questo *stage* e per cui io l'ho scelto. Tratto il ruolo degli *stage* in azienda, il problema che il progetto affronta, il suo posto nella strategia di Halue, gli obiettivi e i vincoli fissati dal piano di lavoro e le mie motivazioni.

2.1 Il ruolo degli *stage* nella strategia aziendale

Halue ospita in media tre o quattro *stage* all'anno, ed è soprattutto attraverso gli *stage* che collabora con le università. Gli stagisti provengono per lo più dall'Università di Padova, dai corsi di laurea triennale e magistrale. L'azienda attribuisce a queste esperienze due scopi:

- svolgere ricerca e sviluppo su temi che i progetti per i clienti non lasciano il tempo di esplorare;
- conoscere persone da inserire nel *team*, valutando al termine di ogni percorso la possibilità di proseguire la collaborazione con lo stagista.

Il primo scopo spiega la forma che gli *stage* assumono. Uno stagista lavora a tempo pieno su un solo tema per circa due mesi, da 300 a 320 ore lavorative, con una continuità che i dipendenti non hanno, perché l'azienda li ripartisce ogni mese tra più clienti (sezione 1.4.1). La quota di tempo che Halue dedica alla ricerca e sviluppo (sezione 1.5) è frammentata, per i dipendenti, tra un progetto e l'altro; in uno *stage* si concentra invece su un unico tema. Halue affida quindi agli stagisti temi nuovi, su cui non ha ancora un'esperienza consolidata, e un *tutor* che li segue ogni giorno. Il secondo scopo spiega perché lo stagista partecipa ai momenti di coordinamento del *team* e ne adotta gli strumenti, dato che l'azienda valuta il suo modo di lavorare oltre ai risultati che produce.

Alcuni degli *stage* che hanno preceduto il mio riguardavano già gli assistenti conversazionali. Uno di questi, svolto alcuni anni prima, aveva progettato un assistente basato su flussi di conversazione predefiniti. In un sistema di questo tipo il progettista definisce in anticipo gli stati della conversazione e i passaggi tra l'uno e l'altro. A ogni stato l'assistente riconosce, con un modello addestrato a classificare le frasi, l'intenzione espressa dall'utente, la associa a uno dei passaggi previsti e gli propone messaggi predefiniti che lo conducono verso una risposta o un'azione. Il comportamento è deterministico, perché a parità di richiesta l'assistente segue sempre lo stesso percorso, e una richiesta che il progettista non ha previsto riceve soltanto una risposta di ripiego.

Un agente basato su un modello linguistico funziona in un altro modo. Il progettista definisce gli strumenti a disposizione e le istruzioni da seguire, e il modello decide a ogni turno quali strumenti invocare e come comporre la risposta. Il comportamento diventa non deterministico, perché la stessa richiesta può ricevere risposte diverse, ma l'agente affronta anche richieste che nessuno ha previsto. Questo cambiamento, insieme agli strumenti di sviluppo basati sull'IA, ha esteso ciò che una persona può realizzare nelle ore di uno *stage*. Rispetto a quel progetto, ChatCart non segue percorsi stabiliti: cerca nel catalogo, compone l'interfaccia e modifica il carrello in base alla conversazione.

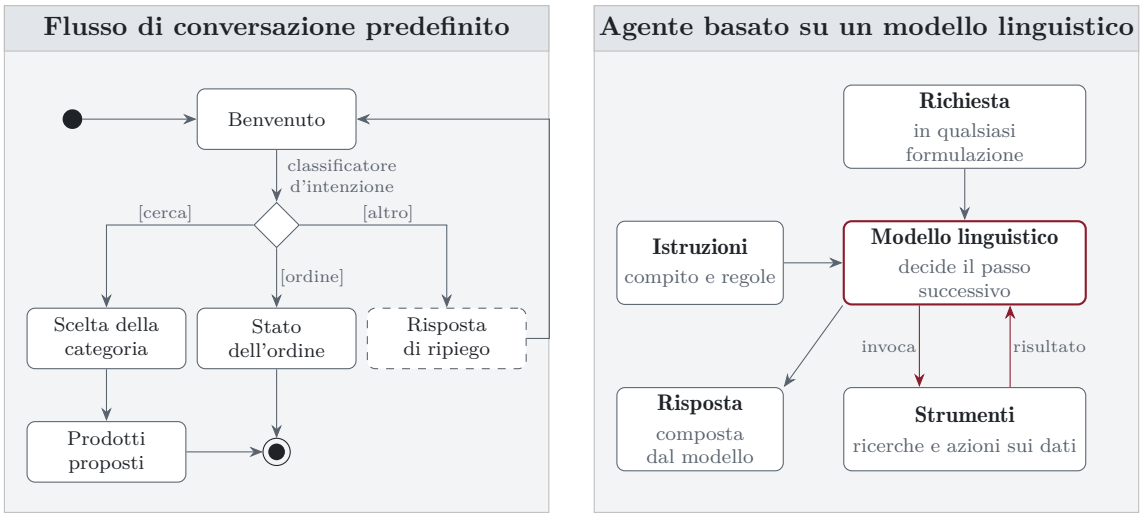


Figura 2.1: A sinistra un esempio di flusso predefinito come diagramma degli stati UML: le guardie tra parentesi quadre sono le intenzioni riconosciute dal classificatore, e ogni richiesta che non rientra tra quelle previste riceve la risposta di ripiego e torna al benvenuto. A destra il ciclo di un agente, in cui il modello invoca gli strumenti e ne legge i risultati finché non dispone di ciò che serve per rispondere.

Ogni *stage* si inserisce in un percorso che lo precede e lo segue. Il tema nasce da un'esigenza interna oppure dall'interesse che può avere per un laureando in infor-

matica. Nel mio caso il percorso è iniziato con la candidatura a luglio 2026 e con un colloquio conoscitivo ad agosto con il *tutor* e con il fondatore. Dopo il colloquio Halue mi ha proposto alcuni temi, tutti di intelligenza artificiale, tra cui:

- uno *storefront* con interfaccia generativa, in cui un agente costruisce la pagina del negozio in tempo reale in base a ciò che il cliente cerca;
- lo sviluppo di un agente interno all'azienda, integrato in profondità con la conoscenza e con gli strumenti di Halue. Il tema si concentrava sulla gestione del contesto dell'agente, cioè sulla scelta delle informazioni da fornire al modello insieme a ogni richiesta, che non possono essere troppe, perché il modello ne elabora una quantità limitata. Per sceglierle serve un sistema di ricerca semantica$_G$, che recupera le informazioni in base al loro significato e non soltanto alle parole che contengono;
- il monitoraggio dei sistemi di IA in produzione, cioè il controllo del comportamento degli agenti già in uso presso l'azienda o i suoi clienti.

I temi coprono tutto il ciclo di un sistema di IA: l'esperienza di chi acquista nei negozi dei clienti di Halue, i processi interni, in cui l'azienda usa già agenti di IA (sezione 1.5), e il funzionamento dei sistemi dopo il rilascio. Ho scelto il primo e l'ho declinato in un assistente di chat, per le ragioni che espongo nella sezione 2.5. ChatCart tocca però, su scala ridotta, anche gli altri due temi:

- l'agente lavora con il contesto del negozio, cioè il catalogo e le pagine informative, che recupera con strumenti dedicati quando la conversazione li richiede;
- le metriche e l'analisi delle conversazioni (OB5, tabella 2.1) sono una prima forma di monitoraggio di un sistema di IA in uso.

Ciascuno dei tre ambiti si presta quindi a un'espansione in profondità: un'interfaccia generativa che compone intere pagine del negozio, un contesto esteso agli ordini e ai dati dei clienti, un monitoraggio che valuta la qualità delle risposte e i costi nel tempo. Ho poi scritto con il *tutor* il piano di lavoro.

Al termine lo stagista presenta il lavoro a tutto il *team* e partecipa a una retrospettiva, simile a quelle con cui Halue chiude gli *sprint* e i progetti per i clienti, in cui si discute che cosa ha funzionato e che cosa correggere. Il lavoro può quindi avere due esiti:

- l'azienda lo archivia come base di conoscenza sul tema;
- l'azienda lo rivede e ne riusa una parte nel codice e nei progetti di produzione.

La scelta dipende dal tipo di progetto e dalla qualità del prodotto. Più avanti descrivo i possibili utilizzi di ChatCart per Halue (sezione 2.3) e le direzioni in cui si può sviluppare (sezione 3.5).

2.2 I limiti degli assistenti conversazionali testuali

Molti negozi *online* offrono ai clienti un *chatbot*$_G$, un programma che conversa con l'utente in linguaggio naturale. Di solito compare come *widget*$_G$, un riquadro sovrapposto alle pagine del negozio che il cliente apre con un pulsante. Questi assistenti rispondono quasi sempre con solo testo. Il testo è adatto alle domande frequenti, ma è poco efficace quando il cliente deve confrontare prodotti, scegliere una taglia o controllare un ordine, cioè nelle operazioni che le pagine di un negozio risolvono con immagini, prezzi, selettori e pulsanti. Un assistente che descrive a parole tre prodotti costringe il cliente ad abbandonare la conversazione per vederli, e per il commerciante questo si traduce in conversazioni che non si concludono con un acquisto. La figura 2.2 mette a confronto le due risposte alla stessa richiesta.

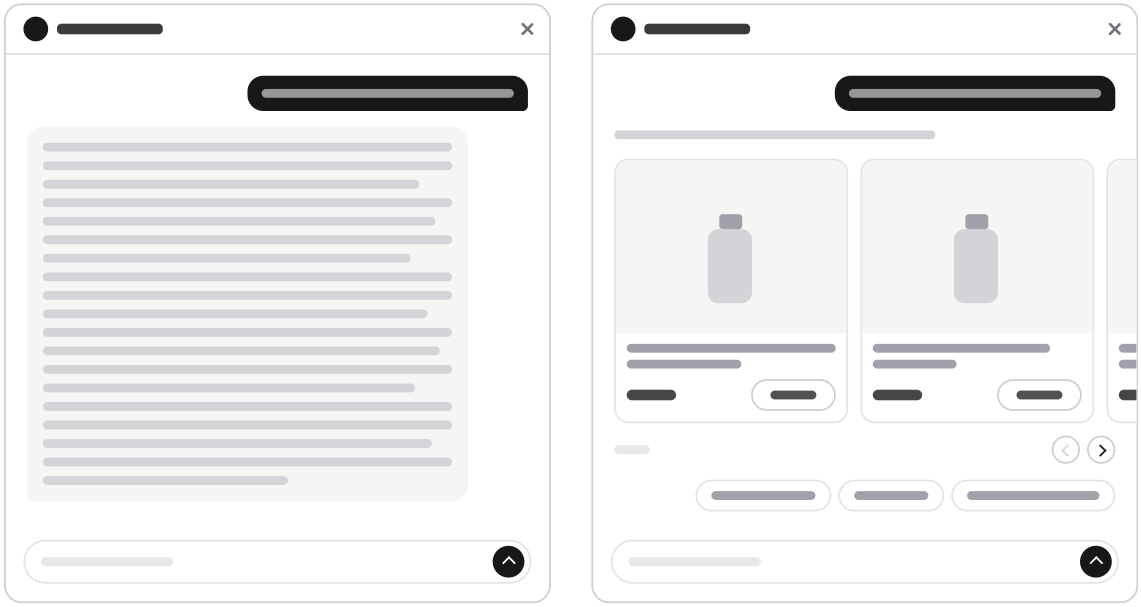


Figura 2.2: *Mockup* della risposta alla stessa richiesta di un assistente testuale, a sinistra, e di ChatCart, a destra.

Il potenziale di un assistente efficace è misurabile. Uno studio sperimentale condotto tra il 2023 e il 2024 su una delle maggiori piattaforme internazionali di commercio elettronico[5] ha confrontato sette applicazioni dell'IA generativa, ciascuna con un gruppo di clienti che la usava e un gruppo di controllo. L'effetto maggiore è quello dell'assistente che risponde alle domande dei clienti prima dell'acquisto,

che ha aumentato le vendite del 16,3%. L'aumento deriva da un numero maggiore di clienti che acquistano, mentre il valore medio del carrello resta invariato. I clienti che traggono più vantaggio dall'assistente sono quelli che conoscono meno la piattaforma.

Le soluzioni disponibili sul mercato appartengono a cinque tipi, e ciascuno lascia scoperta una parte del problema. I primi due sono prodotti di terze parti rivolti a chi acquista:

- **Gli assistenti generici per l'assistenza clienti** risolvono richieste di supporto su più canali a partire dai documenti dell'azienda. Conoscono poco il catalogo e rispondono con testo, quindi sono adatti alla gestione di un reso più che alla scelta di un prodotto.
- **Gli assistenti alla vendita preconfezionati** si installano nel negozio e consigliano prodotti. L'interfaccia, il modello e il comportamento sono decisi dal fornitore e restano uguali per tutti i negozi che li adottano.

Gli altri tre li offre Shopify, da sola o insieme a Google, e non si rivolgono direttamente a chi acquista nel negozio:

- **Sidekick**[6], l'assistente che Shopify integra nel pannello di amministrazione, supporta il commerciante nella gestione del negozio e non si rivolge a chi acquista.
- **Il *server* MCP del negozio**[7], che Shopify attiva per ogni negozio, espone agli agenti la ricerca nel catalogo, il carrello e le condizioni di vendita. È un componente destinato a chi sviluppa un agente, perché fornisce dati e operazioni senza un'interfaccia, e la sua ricerca è la stessa per ogni catalogo.
- **UCP$_{\mathbf{G}}$**[8], uno standard aperto che Shopify e Google hanno pubblicato nel gennaio 2026, permette a un agente di scoprire che cosa offre un negozio e di completare un acquisto. È pensato per gli assistenti di terze parti, per cui la conversazione avviene fuori dal negozio, in un'interfaccia che il marchio non controlla.

Nessuna di queste soluzioni soddisfa insieme i tre requisiti a cui i clienti di Halue attribuiscono più valore:

- l'identità visiva del marchio, perché l'assistente fa parte del negozio e deve avere il suo stesso aspetto;
- i dati specifici del catalogo, con le varianti, le collezioni e le informazioni che distinguono i prodotti di quel marchio;
- un'esperienza d'acquisto che si svolge interamente nel negozio, dalla domanda al carrello.

Un prodotto preconfezionato deve adattarsi a tutti i commercianti, e per questo lascia poco spazio alla personalizzazione. Halue costruisce soluzioni per il singolo cliente, perciò ha scelto di esplorare un assistente su misura basato su un'interfaccia generativa$_G$, cioè un'interfaccia che il modello compone durante la conversazione invece di restare fissata da chi la sviluppa. Il termine comprende approcci che si distinguono per quanto controllo lasciano al modello:

- il modello risponde soltanto con testo, e l'interfaccia resta quella fissa della chat;
- il modello sceglie un componente da una libreria predefinita, come una scheda prodotto, e lo popola in tempo reale con i dati del negozio, mentre aspetto e comportamento del componente restano decisi da chi lo sviluppa;
- il modello genera da sé la struttura dell'interfaccia, per esempio sotto forma di codice o di una descrizione degli elementi da disporre, e può quindi produrre forme che nessuno ha previsto.

Halue ha scelto il secondo approccio. Il marchio conserva il controllo sull'aspetto dei componenti, e il modello decide soltanto quale mostrare e con quali dati. Le scelte del modello restano non deterministiche, ma ognuna produce un componente progettato e verificato in anticipo: un errore riguarda al più il componente o i dati scelti, mentre la grafica e il funzionamento dei pulsanti restano quelli previsti. Il terzo approccio offre più libertà, ma rende difficile garantire la coerenza con il tema del negozio e la correttezza di prezzi, varianti e pulsanti d'acquisto, che in un negozio hanno conseguenze dirette sulle vendite.

2.3 Il commercio agentico nella visione di Halue

Il *server* MCP del negozio e UCP, che ho presentato nella sezione 2.2 tra le soluzioni disponibili, sono due elementi di un cambiamento più ampio, il commercio

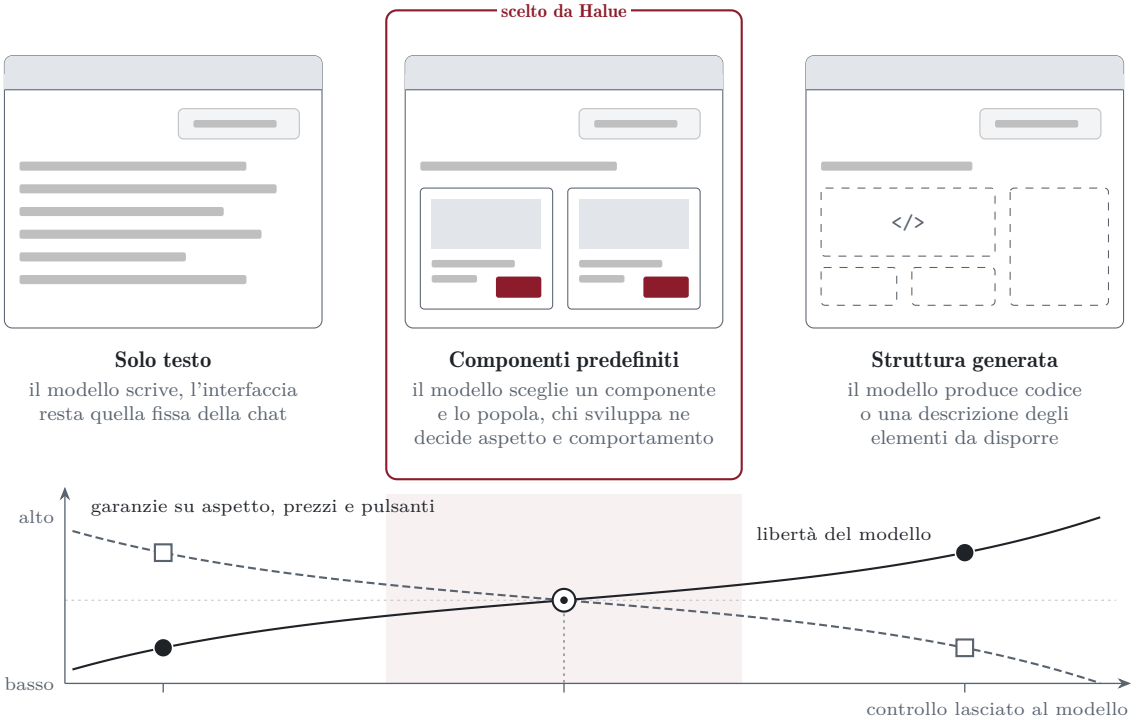


Figura 2.3: I tre approcci all'interfaccia generativa, in ordine di controllo lasciato al modello. Le finestre sono schematiche: in bordeaux i pulsanti d'acquisto, che nel secondo approccio fanno parte del componente e non dipendono dal modello; i riquadri tratteggiati indicano elementi di cui il modello decide forma e posizione. Il grafico è qualitativo e confronta gli approcci senza misurarli: l'approccio scelto da Halue si colloca dove le curve si incrociano, cioè dove la libertà del modello e le garanzie si bilanciano.

agentico$_G$: l'insieme delle esperienze d'acquisto in cui un agente di intelligenza artificiale accompagna il cliente nella ricerca, nella scelta e nell'acquisto di un prodotto. Le soluzioni della sezione 2.2 mostrano le due direzioni in cui questo cambiamento si sviluppa:

- nella prima il negozio si apre agli assistenti generici di terze parti, che lo raggiungono attraverso protocolli come MCP e UCP; il cliente conversa altrove e il negozio fornisce il catalogo e il pagamento;
- nella seconda l'assistente appartiene al marchio e opera dentro il negozio, con la sua interfaccia e con strumenti realizzati attorno al suo catalogo.

La sezione 2.2 spiega perché, per i clienti di Halue, l'assistente deve avere l'aspetto del marchio e conoscerne il catalogo. Le piattaforme promuovono la prima direzione, poiché qualsiasi commerciante la può adottare senza alcuno sviluppo dedicato. In cambio il negozio diventa una fonte di dati per un assistente che il marchio non controlla. È l'assistente a decidere come presentare i prodotti, quali proporre accanto a quello cercato e in quale ordine, e la relazione con il cliente passa per un'interfaccia

esterna. La seconda direzione lascia questa relazione al marchio, ma richiede un progetto su misura per ogni negozio.

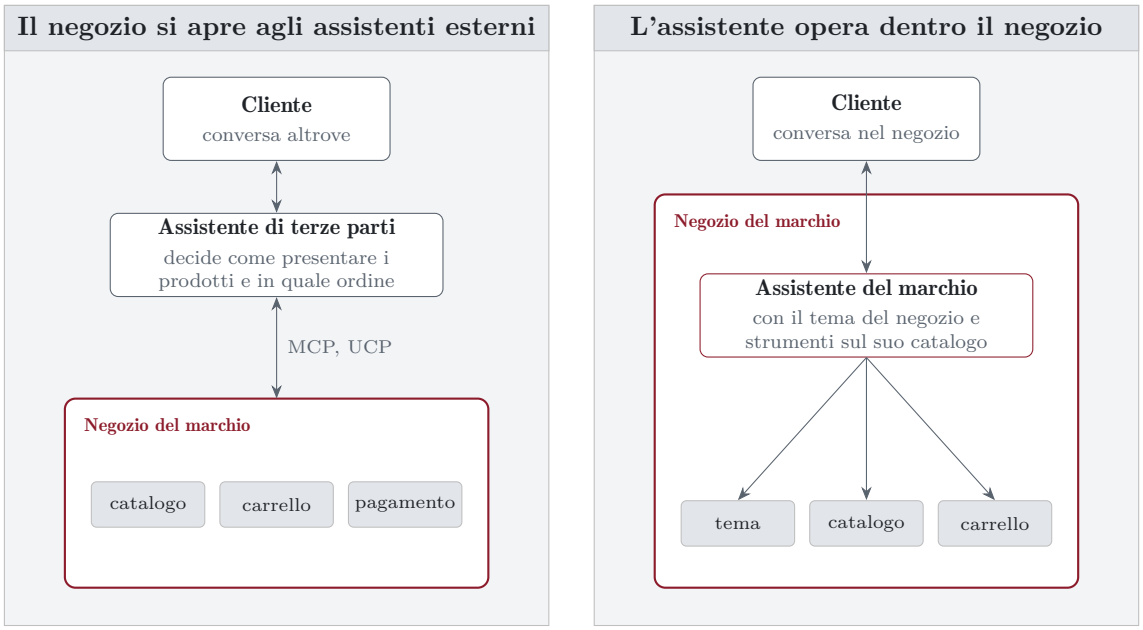


Figura 2.4: Le due direzioni del commercio agentico. Il bordo bordeaux delimita ciò che il marchio controlla: nella prima direzione soltanto i dati e il pagamento, nella seconda anche la conversazione. Nella seconda gli strumenti dell'assistente sono costruiti sul tema e sul catalogo del negozio, mentre il *server* MCP espone le stesse operazioni a qualsiasi agente.

L'assistente su misura della sezione 2.2 appartiene alla seconda direzione. Halue ha scelto di esplorarla con ChatCart, pur seguendo già i nuovi protocolli nei progetti dei clienti: le due direzioni possono convivere, perché lo stesso negozio può esporre il catalogo agli assistenti esterni e offrire ai visitatori un assistente proprio.

Oltre alle ragioni di prodotto esposte nella sezione 2.2, la scelta dipende dal modello di attività dell'azienda. Halue lavora su commessa e non vende prodotti propri (sezione 1.1), quindi innova attraverso le competenze che porta nei progetti dei clienti. Un assistente del marchio si costruisce sul catalogo, sul tema e sui processi di un singolo negozio, cioè sul lavoro che Halue svolge già per i suoi clienti Shopify Plus. In questa direzione un'azienda delle sue dimensioni può offrire un servizio che le piattaforme non coprono, mettendo a frutto la conoscenza approfondita che ha dei negozi dei propri clienti.

Halue sviluppa già agenti di IA collegati ai dati dei negozi e li usa nei propri processi interni (sezione 1.5). Un assistente che conversa con chi visita il negozio pone però problemi propri: deve integrarsi con il tema del negozio, rispondere in pochi secondi e comportarsi in modo prevedibile con qualsiasi interlocutore. Lo *stage* doveva

approfondire questi aspetti su un caso abbastanza circoscritto da poterlo completare nella durata prevista.

Il risultato è per Halue una base di ricerca e sviluppo. L'obiettivo dichiarato è arrivare a proporre eventualmente questo tipo di soluzione ai clienti come servizio, con tempi e forma ancora da stabilire. Il primo passo è stata la presentazione al *team* del 9 novembre 2026, l'ultimo giorno di *stage*: trenta minuti sul prodotto, quindici sulla parte tecnica e quindici di domande. Le domande hanno indicato che cosa interessa a chi riprenderà il lavoro, cioè le scelte di progettazione e architetturali, come la gestione dello stato nella sincronizzazione del carrello, e il modo in cui ho istruito l'agente perché si comportasse come previsto.

2.4 Obiettivi e vincoli del progetto

Il piano di lavoro, che ho scritto con il *tutor* ad agosto 2026, divide il prototipo in due livelli. L'*app layer* è il *widget* di chat nel negozio e si limita a mostrare la conversazione e i componenti. L'*agent layer* è il *backend*, cioè il programma eseguito su un *server*, che ospita l'agente e conserva lo stato della conversazione. Per interrogare le API di Shopify l'agente usa il *tool calling*$_{\text{G}}$, il meccanismo con cui un modello linguistico chiede al programma che lo ospita di eseguire una funzione e ne riceve il risultato.

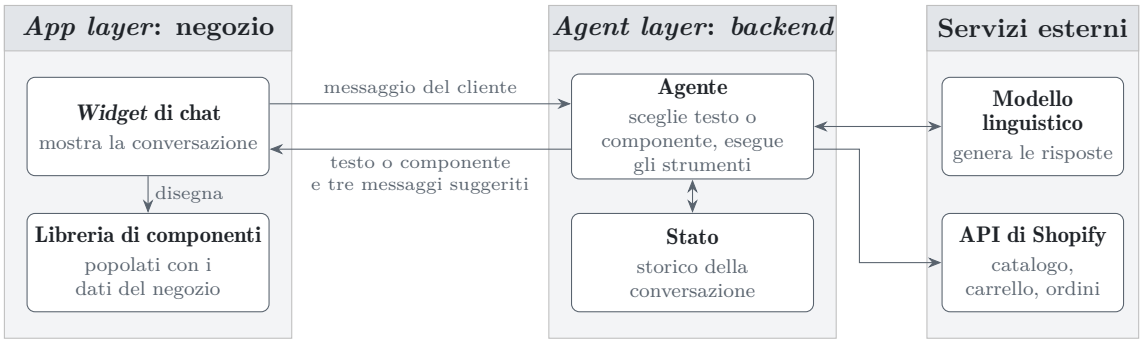


Figura 2.5: L'architettura su due livelli prevista dal piano di lavoro. Il *widget* non contiene logica conversazionale: ogni decisione spetta all'agente, che può interrogare più volte modello e API prima di rispondere.

Ogni obiettivo ha un identificativo, che uso nel resto della relazione, e uno di tre livelli di priorità. Gli obiettivi obbligatori (tabella 2.1) sono vincolanti. Chiedono di dimostrare che l'approccio è fattibile e di misurarne l'efficacia.

ID	Obiettivo
OB1	*App layer*: *widget* di chat in React e TypeScript integrato nel negozio
OB2	*Agent layer*: *backend* Node.js con un agente dotato di *tool calling*, in grado di interrogare le API di Shopify
OB3	Risposte in linguaggio naturale (domande generali, sui prodotti, di chiarimento) oppure con un componente predefinito, scelto e popolato in base alla conversazione
OB4	Libreria di almeno due o tre componenti flessibili e tre messaggi suggeriti per ogni risposta
OB5	Raccolta di metriche di coinvolgimento e di conversione sull'uso del *widget*

Tabella 2.1: Gli obiettivi obbligatori del piano di lavoro.

Gli obiettivi desiderabili hanno un valore aggiunto riconoscibile, quelli facoltativi un valore aggiunto non prioritario (tabella 2.2). Riprendono tre aspetti che il *tutor* ha indicato come interessanti per l'azienda:

- il supporto a più mercati, che riguarda molti clienti di Halue;
- i temi Hydrogen, in cui riusare i componenti React è più semplice e integrarsi con il carrello è più complesso;
- una ricerca nel catalogo scritta su misura, perché il *tutor* giudicava elementare quella del *server* MCP di Shopify.

ID	Priorità	Obiettivo
DE1	desiderabile	Supporto a Shopify Markets$_{G}$, la funzione con cui un negozio vende in più paesi con lingue e valute diverse
DE2	desiderabile	Componente per lo stato dell'ordine e il supporto dopo la vendita
FA1	facoltativo	Prova di fattibilità dell'integrazione su Hydrogen, con il riuso dei componenti del *widget*
FA2	facoltativo	Confronto sperimentale tra la ricerca su misura, il *server* MCP di Shopify e un eventuale *server* MCP proprio

Tabella 2.2: Gli obiettivi desiderabili e facoltativi del piano di lavoro.

Gli obiettivi traducono in prodotto i requisiti della sezione 2.2. OB1 porta l'assistente dentro il negozio, così che l'esperienza d'acquisto vi si svolga per intero. OB2 collega l'agente ai dati reali del catalogo, secondo la prima delle regole che Halue

segue nei progetti di IA (sezione 1.5). OB3 e OB4 realizzano l'interfaccia generativa con componenti predefiniti, che rispettano l'identità visiva del marchio, e DE1 estende lo stesso comportamento ai negozi che vendono su più mercati. OB5 serve a verificare se l'assistente produce il beneficio che lo studio citato nella sezione 2.2 attribuisce a un assistente efficace, cioè più clienti che arrivano all'acquisto.

Per l'azienda il criterio di successo era un prototipo funzionante dall'inizio alla fine su un negozio di esempio, accompagnato da una valutazione onesta di ciò che funziona e di ciò che non funziona. Nella sezione 4.1 valuto in che misura ho soddisfatto ogni obiettivo.

Il progetto aveva sei vincoli, ciascuno con una conseguenza sul lavoro:

- **Quantità di lavoro**: da 300 a 320 ore, al massimo a tempo pieno, dal 14 settembre al 9 novembre 2026. Con una durata fissa dovevo decidere presto che cosa approfondire e che cosa lasciare agli sviluppi futuri.
- **Modalità di lavoro**: da remoto, con un giorno a settimana in sede, un allineamento quotidiano con il *tutor* e una riunione settimanale di un'ora. Il confronto passava quindi soprattutto per brevi riunioni a distanza e per i messaggi su Slack.
- **Tecnologie**: TypeScript in tutto il progetto, dal *widget* al *backend*. Il piano rimandava alle prime settimane la scelta del modello linguistico e degli strumenti per costruire l'agente, che il *tutor* ha lasciato a me.
- **Piattaforma**: prima i temi Liquid, i più diffusi, e Hydrogen soltanto come eventuale approfondimento finale. Il *widget* doveva quindi funzionare all'interno di temi scritti da altri.
- **Ambiente**: un unico negozio di sviluppo, senza clienti né traffico reali, con due temi installati per le prove. L'ho allestito come un negozio di cosmetica botanica con circa quaranta prodotti, perché il comportamento di un assistente dipende dal catalogo su cui opera. Varianti, collezioni, confezioni regalo, prodotti esauriti e archiviati lo avvicinano a un caso reale. Senza traffico, però, le metriche di OB5 vengono soltanto dalle prove.
- **Natura del prodotto**: un prototipo di ricerca e sviluppo, essenziale ma solido, senza i requisiti di scala e di costo di un prodotto in produzione. Per questo non ho ottimizzato il consumo di *token* né la capacità di servire molti negozi insieme.

Il piano ripartisce 310 ore indicative tra dieci attività (figura 2.6) e prevede di rifinire

la ripartizione con il *tutor* nelle prime settimane. Nella sezione 3.1.1 la confronto con il tempo che ho effettivamente impiegato.

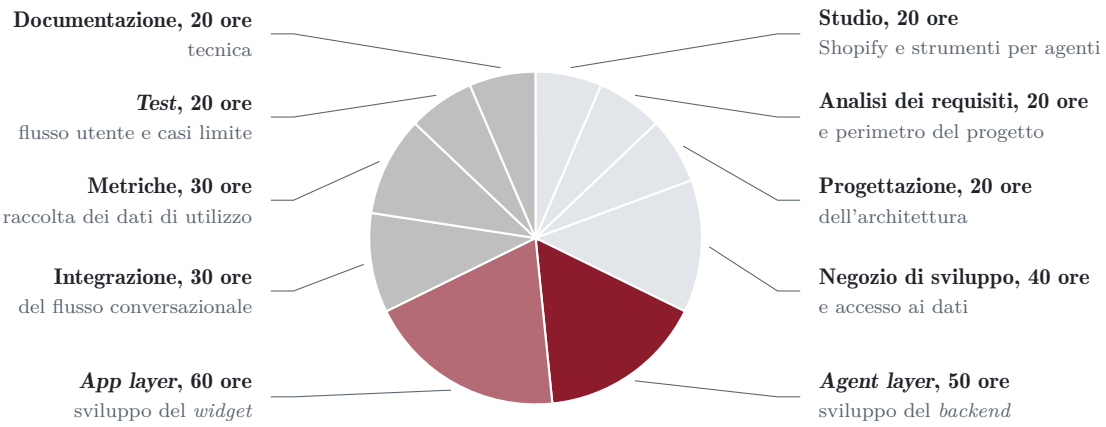


Figura 2.6: La ripartizione delle ore prevista dal piano di lavoro, in senso orario dall'alto. In grigio chiaro le attività che precedono la scrittura del codice, in bordeaux lo sviluppo dei due livelli, in grigio medio le attività che lo completano.

2.5 Motivazioni e obiettivi personali

Da marzo 2026 lavoro in proprio come sviluppatore di negozi Shopify. Il mio cliente principale è Dr Kampes[9], un marchio per cui ho seguito quasi ogni aspetto del negozio: ho creato e configurato il negozio, gestito e ampliato il catalogo, impostato le regole di spedizione, realizzato lo *storefront* e collegato il CRM, con le automazioni successive all'acquisto e quelle di *marketing*. Questo lavoro mi ha dato una visione ampia della piattaforma, ma lasciava una lacuna nella mia esperienza: lo sviluppo di applicazioni su misura e la gestione del negozio attraverso le Admin API di Shopify.

Prima di scegliere Halue ho sostenuto un colloquio con un'altra azienda, esterna alle offerte dell'ateneo, e ho valutato le altre proposte di *stage*. Erano pensate per il lavoro, e per un potenziale inserimento, in aziende grandi e strutturate, mentre io, per esperienza personale, ho sempre prediletto gli ambienti piccoli e dinamici, in cui posso sperimentare creativamente e procedere per iterazioni rapide, come quelli

delle *startup*$_{\text{G}}$, le giovani imprese nate per sviluppare un prodotto innovativo. La proposta di Halue rispondeva a questa preferenza e offriva tre vantaggi:

- riguardava Shopify, la piattaforma su cui già lavoravo, in un'azienda che la usa ogni giorno, e mi permetteva di passare dall'uso dei suoi strumenti alla costruzione di applicazioni;
- mi garantiva due mesi a tempo pieno su tecnologie che già mi interessavano, un'occasione che il lavoro in proprio concede di rado;
- prevedeva il lavoro soprattutto da remoto, che Halue mi ha proposto già al colloquio. Per me era una condizione decisiva: il lavoro da remoto corrisponde al mio modo di lavorare, e intendo privilegiarlo anche nei lavori futuri per la mobilità e le scelte di vita che consente.

Tra i temi proposti ho scelto l'interfaccia generativa. La gestione del contesto mi interessava molto e apriva diverse strade di integrazione con altri strumenti e sistemi, ma comportava un rischio. Aziende come Anthropic[10] e OpenAI[11] aggiornano i propri assistenti a ritmo serrato, e vi aggiungono una dopo l'altra funzioni come la ricerca nei documenti e negli strumenti di un'azienda. Un sistema costruito in due mesi poteva diventare superfluo con un solo aggiornamento di un assistente generico. Un assistente di vendita nel negozio di un marchio dipende invece da elementi che solo quel negozio possiede, cioè il catalogo, il tema e il carrello, e un assistente generico lo sostituisce con più difficoltà. Risponde inoltre a un problema di conversione, con metriche misurabili (sezione 2.2), ed è più semplice da offrire ai clienti come servizio personalizzato.

L'idea dell'interfaccia generativa è di Halue, che la proponeva come uno *storefront* costruito in tempo reale da un agente. Vi ho riconosciuto un'esigenza di Dr Kampes, che mi aveva parlato di un assistente capace di aiutare i visitatori del negozio e di indirizzarli verso l'acquisto. Ho quindi proposto di declinare il tema in un assistente di chat che sceglie e popola con i dati del negozio componenti come schede prodotto, tabelle di confronto, guide alle taglie e lo stato di un ordine, e che accompagna il cliente dalla scoperta dei prodotti all'assistenza dopo la vendita. Un assistente di questo tipo si aggiunge al tema esistente di un negozio, e si applica quindi anche ai negozi che Halue e io, su scala diversa, seguiamo già.

Il tema toccava anche altri miei interessi:

- imparare a sviluppare applicazioni Shopify, per estendere gli *storefront* con soluzioni su misura;
- collegare i negozi ai miei agenti di IA attraverso le Storefront API e le Admin API (sezione 1.3);
- conoscere la piattaforma a fondo, capendo come si incastrano i suoi componenti, per costruire da zero ciò che finora avevo soltanto usato.

Volevo infine mettere alla prova in un progetto reale ciò che ho imparato nel corso di studi: i *design pattern*$_G$, cioè soluzioni riconosciute a problemi ricorrenti di progettazione, il rigore nella pianificazione e una progettazione che facilita le estensioni future e la manutenzione.

Da queste ragioni ho ricavato quattro obiettivi personali (tabella 2.3), che nella sezione 4.1 valuto insieme a quelli del piano.

ID	Obiettivo
OP1	Estendere le mie competenze su Shopify dai temi allo sviluppo di applicazioni
OP2	Acquisire esperienza nella progettazione di sistemi basati su agenti
OP3	Ricavare dal progetto conoscenze e soluzioni riutilizzabili nella mia attività professionale
OP4	Gestire in autonomia il lavoro da remoto, dove nessuno segue la mia giornata in modo continuo e spetta a me organizzarla, stabilire le priorità e prevedere le pause

Tabella 2.3: I miei obiettivi personali per lo *stage*.

Questo capitolo e il precedente definiscono il contesto del progetto: l'azienda che lo ha proposto, il problema che affronta, gli obiettivi e i vincoli fissati dal piano di lavoro e le ragioni per cui l'ho scelto. Il progetto di *stage* nasce dall'incontro di tre esigenze: Halue voleva svolgere una ricerca propria sul commercio agentico, gli assistenti disponibili sul mercato rispondono soltanto con testo o scendono a compromessi significativi sull'identità del marchio e sui dati del catalogo, e io cercavo l'occasione di approfondire l'ecosistema Shopify, dallo sviluppo di applicazioni su misura con le Storefront API e le Admin API alla progettazione di agenti basati su modelli linguistici. Nel capitolo 3 racconto la progettazione e lo sviluppo di ChatCart, dalle prime settimane di pianificazione alla presentazione al *team*.

3 Progetto e sviluppo di ChatCart

Racconto lo *stage* come un piccolo progetto a sé stante: il metodo di lavoro, l'analisi, la progettazione, la realizzazione, le sperimentazioni, la verifica e i risultati.

3.1 Metodo di lavoro

3.1.1 Pianificazione e interazione con il *tutor* aziendale

Confronto le modalità di interazione previste dal piano di lavoro con quelle effettive: le riunioni quotidiane, i resoconti di fine giornata su Slack, la riunione settimanale, i confronti su richiesta e le giornate in sede. Discuto le difficoltà della fase iniziale e il modo in cui le ho superate. Descrivo infine i resoconti settimanali al relatore e la svolta della terza settimana: con gli obiettivi obbligatori già soddisfatti, io e il *tutor* abbiamo deciso di dedicare il resto dello *stage* alla sperimentazione. Un diagramma temporale mette a confronto la pianificazione originaria del piano di lavoro con il modo in cui ho usato davvero il tempo, attività per attività.

3.1.2 Organizzazione del lavoro e tracciamento delle attività

Illustro come ho organizzato il mio lavoro, svolto in gran parte da remoto: come individuavo le attività e ne stabilivo le priorità, come ne tenevo traccia e come preparavo le modifiche più estese prima di realizzarle.

3.1.3 Strumenti e assistenti di programmazione

Presento gli strumenti che ho impiegato per lo sviluppo, la verifica e il rilascio, tra cui gli assistenti di programmazione basati su modelli linguistici. Di questi descrivo l'uso che ne ho fatto: la sperimentazione e la validazione rapida delle idee, fino a prototipi funzionanti nell'arco di una giornata, a cui seguiva l'implementazione

definitiva scritta a mano; la stesura dei documenti di progettazione, per individuare in anticipo i casi limite e i *test* necessari; le attività di supporto, come la generazione dei dati di prova. Ciò che ho imparato da questo uso lo discuto nella sezione 4.2.

3.2 Studio preliminare e analisi

3.2.1 L'ecosistema delle applicazioni Shopify

Descrivo la fase iniziale dello *stage*, dedicata allo studio del funzionamento di un'applicazione Shopify (autenticazione, *app proxy*, estensioni del tema, differenze tra Storefront API e Admin API) e degli strumenti per la costruzione di agenti. Distinguo le conoscenze che avevo già acquisito lavorando sui temi Liquid da quelle che ho dovuto costruire per la prima volta.

3.2.2 Un'analisi dei requisiti iterativa

Motivo la scelta, condivisa con il *tutor*, di non svolgere un'analisi dei requisiti completa all'inizio del progetto: il lavoro aveva carattere esplorativo, nessun cliente ne dettava i requisiti, e una specifica fissata a priori sarebbe diventata presto obsoleta. Descrivo il procedimento che ho seguito: a partire dagli obiettivi del piano ho abbozzato i casi d'uso principali e ne ho ricavato i requisiti, che ho rivisto a ogni iterazione insieme al prodotto. Un diagramma mostra i casi d'uso nella loro forma finale e una tabella riporta i requisiti a cui sono arrivato, raggruppati per area, ciascuno con un identificativo e l'obiettivo del piano da cui deriva; ne misuro la copertura nella sezione 3.7.2.

3.3 Architettura del sistema

3.3.1 Visione d'insieme

Illustro l'architettura di ChatCart con un diagramma: il *widget* nel negozio (*app layer*), il *server* che ospita l'agente (*agent layer*), la *dashboard* del commerciante e i servizi esterni. Motivo la scelta di uno stile funzionale anziché orientato agli oggetti, la decisione, su indicazione del *tutor*, di non introdurre astrazioni prima che fossero necessarie e i *design pattern* che ho adottato quando lo sono diventate.

3.3.2 Il contratto tra agente e *widget*

Approfondisco la scelta su cui si regge l'interfaccia generativa: un unico elenco di strumenti, condiviso tra agente e *widget*, che distingue gli strumenti che leggono i dati del negozio da quelli che mostrano un componente e chiudono la risposta. Spiego come da questo elenco derivo i tipi condivisi tra le due parti, così che il *widget* sappia sempre disegnare ciò che l'agente sceglie.

3.4 Realizzazione

3.4.1 L'agente conversazionale

Descrivo il processo con cui l'agente costruisce una risposta: un ciclo in cui il modello invoca gli strumenti per consultare il catalogo, finché non seleziona un componente da mostrare. Discuto come ho ripartito il controllo del comportamento tra le istruzioni al modello e le regole fisse nel codice, come ho generato i tre messaggi suggeriti e come ho scelto il modello e il livello di ragionamento, confrontandone velocità e accuratezza. Un diagramma di sequenza segue il percorso di un messaggio dall'utente alla risposta. Questa parte risponde agli obiettivi OB2 e OB3.

3.4.2 Il *widget* e l'integrazione con il tema del negozio

Presento il *widget*: i componenti (schede prodotto con selezione della variante, schede con le risposte tratte dalle pagine del negozio, collegamenti alle collezioni), il supporto a più lingue e valute e, in particolare, la sincronizzazione del carrello con il tema del negozio. Analizzo perché ogni tema Shopify gestisce il carrello in modo diverso e come ho affrontato il problema con un *adapter* per ciascun tema e una soluzione di ripiego. Questa parte risponde agli obiettivi OB1, OB4 e DE1.

3.4.3 Metriche di utilizzo e analisi delle conversazioni

Descrivo gli eventi che il *widget* registra, il *funnel* che la *dashboard* presenta al commerciante e l'analisi delle conversazioni con Jev, un modello decisionale di TypeSafe AI uscito durante lo *stage*: a differenza di un modello linguistico generativo, non produce testo ma restituisce decisioni tipizzate con la loro probabilità. Da qui apro il confronto tra i due tipi di modello nei compiti decisionali. Questa parte risponde all'obiettivo OB5.

3.5 Sperimentazioni e sviluppi futuri

Dedico questa sezione alle funzionalità che ho sperimentato senza portarle nel prodotto finale, perché documentano l'orientamento alla ricerca che io e il *tutor* abbiamo dato al progetto dalla terza settimana: i messaggi di invito che comparivano prima dell'apertura della chat, i tre tentativi di adattare l'aspetto del *widget* al tema del negozio, l'impiego di Jev nella scelta dei componenti, il pannello a tutto schermo su *desktop*. Per ciascuna espongo l'ipotesi di partenza, l'esito che ho osservato e le ragioni per cui io e il *tutor* abbiamo deciso di non inserirla nel prodotto consegnato. Da queste prove ricavo infine le strade lungo cui ChatCart potrebbe evolvere in futuro.

3.6 Verifica e validazione

Illustro la mia strategia di verifica: *test* automatici concentrati sui punti in cui ChatCart interagisce con sistemi esterni che non controllo, eseguiti su una base di dati reale e senza simulare i moduli interni, e le ragioni di questa scelta. Discuto i limiti dei *test* tradizionali applicati a un agente che non risponde mai due volte nello stesso modo. Concludo con la validazione svolta insieme al *tutor* e con il rilascio dell'applicazione *online*.

3.7 Risultati

3.7.1 Il prodotto dal punto di vista dell'utente

Presento ChatCart dal punto di vista di chi lo utilizza, attraverso una sequenza di schermate: un cliente che cerca un prodotto, lo confronta con altri, lo aggiunge al carrello e raggiunge il pagamento; poi il commerciante che consulta il *funnel* e le conversazioni nella *dashboard*. Dichiaro inoltre i limiti noti del prototipo.

3.7.2 Risultati quantitativi

Riporto in tabelle i dati quantitativi del progetto: la copertura dei requisiti definiti nella sezione 3.2.2, la copertura dei *test* con le scelte che la motivano, le righe di codice per ciascuna parte dell'applicazione e i documenti che ho consegnato al *team*.

4 Valutazione retrospettiva

Ripercorro lo *stage* a posteriori: in che misura ho raggiunto gli obiettivi, che cosa ho imparato e quanto la formazione universitaria mi abbia preparato al lavoro che ho svolto.

4.1 Soddisfacimento degli obiettivi

Valuto, sulla base di dati di fatto, il grado di soddisfacimento degli obiettivi presentati nella sezione 2.4 e nella sezione 2.5: una tabella riporta ciascun obiettivo del piano, da OB1 a FA2, e ciascuno dei miei obiettivi personali, con il relativo stato e l'evidenza che lo documenta. Valuto allo stesso modo i prodotti attesi: quelli elencati nel piano di lavoro e la presentazione formale al *team* che il *tutor* mi ha chiesto a fine *stage*. Discuto gli scostamenti, come gli obiettivi che io e il *tutor* abbiamo scartato e i limiti delle metriche raccolte.

4.2 Maturazione professionale

Descrivo la mia maturazione professionale in termini di conoscenze, come l'ecosistema delle applicazioni Shopify e la costruzione di agenti, e di abilità, come il lavoro da remoto, la comunicazione con il *tutor*, la gestione del tempo e la capacità di stabilire cosa escludere dal progetto. Rifletto inoltre su ciò che ho appreso dall'uso dell'intelligenza artificiale nella programmazione, il tema su cui si è concentrato l'incontro conclusivo con il fondatore.

4.3 Formazione universitaria e lavoro

Confronto le competenze che lo *stage* mi ha richiesto con quelle che ho acquisito nel corso di studi. Individuo quelle che si sono rivelate utili e le lacune che sono emerse, come la progettazione di agenti e la verifica di sistemi dal comportamento non deterministico, sia rispetto allo *stage* sia rispetto all'idea che ho del mondo del lavoro.

Acronimi e abbreviazioni

Elenco in ordine alfabetico gli acronimi e le abbreviazioni introdotti nei capitoli 1 e 2. Quando presente, il collegamento rimanda all'approfondimento nel glossario.

ADR: *Architecture Decision Record*.

API: *Application Programming Interface*.

AWS: Amazon Web Services.

B2B: *Business-to-Business*.

B2C: *Business-to-Consumer*.

CMS: *Content Management System*.

CRM: *Customer Relationship Management*.

ERP: *Enterprise Resource Planning*.

FAQ: *Frequently Asked Questions*.

IA: intelligenza artificiale.

iPaaS: *Integration Platform as a Service*.

LLM: *Large Language Model*.

MCP: *Model Context Protocol*.

SAL: stato di avanzamento dei lavori.

SemVer: *Semantic Versioning*.

UCP: *Universal Commerce Protocol*.

UI: *User Interface.*

UX: *User Experience.*

Glossario

Raccolgo in ordine alfabetico i termini tecnici introdotti nel testo. Alla prima occorrenza nel corpo della relazione, il termine e il pedice G rimandano alla voce corrispondente.

ADR Documento che registra una decisione di architettura: il contesto in cui nasce, le alternative considerate, la scelta e le sue conseguenze. Conserva le motivazioni della scelta anche quando cambia chi lavora al progetto.

API Interfaccia con cui un programma rende disponibili dati o operazioni ad altri programmi, secondo un contratto definito. In Shopify, la Admin API permette di amministrare il negozio; la Storefront API espone le funzioni rivolte all'esperienza d'acquisto.

B Impact Assessment Questionario gratuito con cui un'azienda misura il proprio impatto sociale e ambientale in cinque aree: *governance*, lavoratori, clienti, comunità e ambiente. Il punteggio che ne risulta è anche la base per la certificazione B Corp.

B2B Modello commerciale in cui un'azienda vende prodotti o servizi ad altre aziende. Nel commercio elettronico può richiedere cataloghi, listini e condizioni di acquisto diversi per ciascun cliente.

B2C Modello commerciale in cui un'azienda vende prodotti o servizi al consumatore finale. È il contesto dell'interfaccia conversazionale di ChatCart, rivolta a chi acquista nel negozio.

Chatbot Programma che conversa con l'utente in linguaggio naturale, per iscritto o a voce. Nei negozi *online* risponde alle domande dei visitatori; quelli più recenti si basano su un modello linguistico e possono consultare i dati del negozio.

Commercio agentico Insieme delle esperienze d'acquisto in cui un agente di intelligenza artificiale accompagna il cliente, o agisce per suo conto, nella ricerca, nella scelta e nell'acquisto di un prodotto. L'agente può appartenere al negozio oppure a una terza parte che raggiunge il negozio tramite protocolli come MCP e UCP.

CRM Insieme di pratiche e sistemi per gestire le relazioni di un'azienda con i propri clienti. Un sistema CRM raccoglie, per esempio, contatti, interazioni commerciali e richieste di assistenza.

Design pattern Soluzione riconosciuta e documentata a un problema di progettazione che si ripresenta in contesti diversi, descritta in modo indipendente dal linguaggio. Ne è un esempio l'*adapter*, che rende compatibili due interfacce senza modificarle.

Design system Insieme coerente di principi, regole visive e componenti riusabili per progettare un'interfaccia. Comprende elementi come colori, tipografia, spaziature e comportamenti dei componenti, così da mantenere riconoscibile l'identità del prodotto o del marchio.

E-commerce Commercio di prodotti o servizi attraverso canali digitali. Un negozio elettronico comprende il catalogo, la selezione dei prodotti, il carrello e il pagamento, oltre ai collegamenti con i sistemi che gestiscono ordini e logistica.

ERP Sistema gestionale che integra dati e processi di diverse aree aziendali, come amministrazione, acquisti, magazzino e vendite. Il collegamento con il negozio permette, per esempio, di sincronizzare ordini e disponibilità dei prodotti.

Founder & Senior Consultant Ruolo che unisce la fondazione dell'azienda all'attività di consulenza esperta. In Halue lo ricopre Valentino Baraldo.

GraphQL Linguaggio di interrogazione per API e insieme di regole per eseguire le richieste su uno schema di dati tipizzato. Il programma che invia la richiesta specifica i campi di cui ha bisogno; Shopify lo adotta per la Admin API e la Storefront API.

Headless Architettura che separa la gestione dei dati e delle funzioni applicative dall'interfaccia che li presenta. In un negozio di questo tipo l'interfaccia è un'applicazione autonoma che comunica con la piattaforma commerciale tramite API; in un CMS la stessa separazione riguarda i contenuti e la loro presentazione.

Hydrogen *Framework* di Shopify basato su React per costruire interfacce di negozi *headless*. Offre strumenti e componenti per accedere ai dati del negozio tramite la Storefront API; Oxygen è il servizio di Shopify che può ospitare queste applicazioni.

Interfaccia generativa Interfaccia in cui un modello linguistico decide, a ogni risposta, quali elementi mostrare all'utente oltre al testo. In ChatCart il modello sceglie tra componenti predefiniti e ne fornisce i dati; l'aspetto dei componenti resta fissato dal codice.

iPaaS Piattaforma erogata come servizio per collegare applicazioni e sistemi diversi. Offre connettori e strumenti per trasformare e trasferire i dati, riducendo la necessità di costruire un'integrazione distinta per ogni coppia di sistemi.

Junior Developer Sviluppatore all'inizio del proprio percorso professionale, che realizza e verifica funzionalità con il supporto di colleghi più esperti.

Landing page Pagina di atterraggio, costruita per accogliere chi arriva da una campagna, da un annuncio o da un collegamento esterno e guidarlo verso un'azione, come l'acquisto di un prodotto. Una *landing page* dinamica cambia contenuti in base a dati come la collezione, il mercato o il visitatore.

Liquid Linguaggio di *template* usato nei temi Shopify. Permette di combinare la struttura delle pagine con i dati del negozio, attraverso variabili, condizioni e cicli; la piattaforma elabora i modelli e genera le pagine da inviare al navigatore.

LLM Modello linguistico di grandi dimensioni, addestrato su ampie raccolte di dati per elaborare e generare testo. In un agente può selezionare strumenti esterni di cui l'applicazione esegue le operazioni; il collegamento a questi strumenti gli permette di usare dati aggiornati che non fanno parte del suo addestramento.

MCP Protocollo aperto che standardizza la comunicazione tra applicazioni basate su modelli linguistici e servizi esterni. Un *server* MCP può esporre strumenti, risorse e istruzioni; è l'applicazione che ospita il modello a gestire il collegamento e l'esecuzione delle richieste.

Project Management Assistant Figura che supporta la gestione dei progetti, contribuendo all'organizzazione delle attività, all'aggiornamento della documentazione e al monitoraggio delle scadenze.

Project/service manager Responsabilità operativa di gestione di un progetto o di un servizio continuativo. Nel metodo di Halue cura la pianificazione delle attività e la capacità disponibile del gruppo di lavoro.

Pull request Proposta di integrare un insieme di modifiche da un *branch* a un altro in un sistema di versionamento. Raccoglie le differenze nel codice, la discussione di revisione e gli esiti dei controlli automatici prima dell'integrazione.

SAL Stato di avanzamento dei lavori. Nel contesto descritto indica un rapporto periodico che riassume il progresso del progetto, le scadenze, i rischi e i punti aperti, per condividere la situazione con le persone coinvolte.

Redesign Riprogettazione grafica e funzionale di un sito o di un negozio esistente, che ne rinnova l'interfaccia e i percorsi dell'utente, spesso senza cambiare la piattaforma su cui si basa.

Ricerca semantica Ricerca che confronta il significato della richiesta con quello dei documenti, invece delle sole parole in comune. Di solito si basa su rappresentazioni numeriche dei testi, dette *embedding*, in cui testi di significato simile risultano vicini.

Scrum *Framework* per affrontare problemi complessi attraverso cicli brevi di lavoro, chiamati *sprint*, e un confronto regolare sui risultati. Prevede responsabilità, eventi e artefatti definiti; nella sezione 1.4.1 distinguo questo riferimento dall'adattamento che Halue ne fa per il lavoro su commessa.

SemVer Convenzione di versionamento semantico che esprime una versione nella forma `MAJOR.MINOR.PATCH`. Rispetto all'API pubblica dichiarata, il primo numero cambia per modifiche incompatibili, il secondo per nuove funzionalità compatibili e il terzo per correzioni compatibili.

Senior Project Manager Figura con esperienza nella gestione dei progetti, responsabile del coordinamento di pianificazione, risorse e avanzamento delle attività.

Senior Software Engineer Professionista dello sviluppo del *software* con esperienza consolidata, che affronta problemi progettuali complessi e contribuisce alle scelte tecniche e alla revisione del codice.

Shopify Plus Proposta di Shopify per le aziende di grandi dimensioni, con funzioni aggiuntive rispetto ai piani standard, tra cui quelle per la vendita B2B.

Shopify Markets Funzione di Shopify per vendere in più paesi dallo stesso negozio. Per ogni mercato si definiscono lingua, valuta, prezzi e indirizzi delle pagine.

Skill Insieme riusabile di istruzioni e, quando necessari, risorse o strumenti, che guida un assistente basato su IA in un'attività specifica. Serve a rendere esplicito un procedimento, ma non garantisce da solo la correttezza dell'esecuzione.

Società Benefit Società che affianca allo scopo di lucro una o più finalità di beneficio comune indicate nello statuto. La disciplina italiana richiede una gestione che bilanci questi obiettivi e una relazione annuale sull'impatto prodotto; la qualifica non coincide con una certificazione privata di sostenibilità.

Software Engineer Professionista che applica metodi di ingegneria del *software* alla progettazione, realizzazione, verifica e manutenzione di sistemi *software*.

Solution & Technical Architect Figura che traduce le esigenze applicative in una soluzione tecnica, definendo l'architettura del sistema, le tecnologie e le integrazioni tra componenti.

Sprint Ciclo di lavoro di durata fissa, non superiore a un mese nel riferimento Scrum, durante il quale il gruppo persegue un obiettivo concordato. La revisione dei risultati e la retrospettiva permettono di adattare il prodotto e il modo di lavorare al ciclo successivo.

Staging Ambiente separato dalla produzione in cui collaudare una versione dell'applicazione prima di renderla disponibile agli utenti. Riproduce le condizioni di esercizio per quanto necessario alla verifica, senza coincidere con il servizio effettivamente usato dai clienti.

Startup Impresa giovane, di solito piccola, nata per sviluppare un prodotto o un servizio innovativo e per verificarne in fretta l'interesse sul mercato, con cicli di sviluppo brevi e ruoli poco formalizzati.

Stand-up Breve riunione quotidiana, nata nei metodi agili, in cui ogni membro del gruppo riferisce su cosa ha lavorato, su cosa lavorerà e quali ostacoli incontra. Il nome viene dall'abitudine di tenerla in piedi, per mantenerla breve.

Storefront Interfaccia di un negozio digitale rivolta al cliente, attraverso cui consultare il catalogo, selezionare prodotti e avviare l'acquisto. In Shopify può basarsi sui temi della piattaforma oppure su un'applicazione separata con architettura *headless*.

Story point Unità di stima relativa che un gruppo usa per confrontare le attività in termini di complessità, impegno e incertezza. Non corrisponde a un numero fisso di ore e acquista significato rispetto alle stime dello stesso gruppo; la scala di Fibonacci è una possibile convenzione, non un obbligo di Scrum.

Team lead Responsabilità di guida tecnica di un gruppo di lavoro. In Halue comprende l'analisi funzionale, le stime per il cliente e il coordinamento tecnico degli sviluppatori.

Ticket Unità di lavoro registrata in un sistema di tracciamento, con un identificativo, una descrizione e uno stato. Può rappresentare una funzionalità, una modifica o un difetto, e raccogliere stime, discussioni e collegamenti alle modifiche del codice.

Token Unità minima di testo elaborata da un modello linguistico: una parola breve, una parte di parola, un segno di punteggiatura. Il modello legge e genera testo un *token* alla volta; la lunghezza del contesto e il costo di utilizzo si misurano in *token*.

Tool calling Meccanismo con cui un modello linguistico chiede al programma che lo ospita di eseguire una funzione, detta strumento, indicandone il nome e gli argomenti. Il programma esegue la funzione e restituisce il risultato al modello, che lo usa per proseguire.

UCP Protocollo aperto per rendere interoperabili i servizi commerciali e gli agenti che operano per conto dell'utente. Definisce modalità condivise per scoprire le capacità di un negozio e interagire con le fasi dell'acquisto; riguarda il dominio commerciale, mentre MCP offre un meccanismo generale per collegare applicazioni e strumenti.

UX/UI Digital Consultant Consulente che progetta l'esperienza utente e l'interfaccia di un prodotto digitale, considerando percorsi d'uso, struttura delle informazioni e aspetto visivo.

Widget Elemento dell'interfaccia autonomo, inserito in una pagina che non lo ha prodotto. Il *widget* di ChatCart è il riquadro di chat sovrapposto alle pagine del negozio.

Riferimenti

1. Halue S.r.l., sito istituzionale, https://halue.com/it, pagine «Chi siamo», «Approccio» e «Progetti», consultate il 4 novembre 2026.
2. Halue S.r.l., «Società Benefit», https://halue.com/it/societa-benefit, consultata il 10 novembre 2026.
3. Shopify, «Shopify Plus», https://www.shopify.com/plus, consultata il 10 novembre 2026.
4. Shopify, «Halue Srl», elenco dei partner di Shopify, https://www.shopify.com/it/partners/directory/partner/halue-srl, consultata il 10 novembre 2026.
5. L. Fang, Z. Yuan, K. Zhang, D. Donati, M. Sarvary, «Generative AI and Sales Productivity: Field Experiments in Online Retail», arXiv:2510.12049, 2025, https://arxiv.org/abs/2510.12049, consultato il 10 novembre 2026.
6. Shopify, «Sidekick», Centro assistenza, https://help.shopify.com/en/manual/ai-powered-tools/sidekick, consultato il 10 novembre 2026.
7. Shopify, «Storefront MCP», documentazione per sviluppatori, https://shopify.dev/docs/apps/build/storefront-mcp, consultato il 10 novembre 2026.
8. Shopify e Google, *Universal Commerce Protocol*, https://ucp.dev e https://www.shopify.com/it/ucp, consultati il 10 novembre 2026.
9. Dr Kampes, sito del negozio, https://drkampes.com, consultato l'11 novembre 2026.
10. Anthropic, «Claude and your productivity platforms», 16 ottobre 2025, https://www.anthropic.com/news/productivity-platforms, consultato l'11 novembre 2026.
11. OpenAI, «Introducing company knowledge», ottobre 2025, https://openai.com/index/introducing-company-knowledge/, consultato l'11 novembre 2026.